# AI Agent Canvas

Multi-Agent Enterprise Copilot Framework

Build intelligent AI copilots with .NET 9 and CopilotKit.

Orchestrate specialized agents that reason, plan, and act through a shared tool registry.

Complete Reference Guide

# Table of Contents

# Overview

Key concepts behind AI agents, the Microsoft Agent Framework, tools, MCP, context providers, and the AG-UI streaming protocol.

# What Are AI Agents

AI agents represent a fundamental shift from traditional chatbots and simple LLM wrappers. Where a chatbot follows scripted flows and an LLM wrapper sends a prompt and returns a response, an AI agent autonomously reasons, plans, and takes action to accomplish goals.

## Chatbots vs LLM Wrappers vs AI Agents

| Capability | Traditional Chatbot | LLM Wrapper | AI Agent |
|---|---|---|---|
| Natural language | Rule-based / intent matching | Full LLM capability | Full LLM capability |
| Decision making | Scripted decision trees | Single inference call | Multi-step reasoning |
| Tool use | None or hardcoded | None | Dynamic tool selection |
| Memory | Session state only | Stateless | Persistent context |
| Autonomy | None | None | Goal-directed behavior |

## Key Characteristics of AI Agents

- **Autonomy** -- Agents decide which actions to take without step-by-step human instruction. Given a goal, they determine the path.
- **Tool Use** -- Agents interact with external systems through defined tools: APIs, databases, file systems, and other services.
- **Reasoning** -- Agents analyze information, break complex problems into steps, and adapt their approach based on intermediate results.
- **Memory and Context** -- Agents maintain context across interactions including conversation history, retrieved documents (RAG), user preferences, and domain-specific knowledge.

## The Agent Loop

Every AI agent follows a fundamental execution cycle: **Perceive** (receive input, gather context) then **Reason** (analyze input, plan next step) then **Act** (call a tool or generate response) then **Observe** (process result, decide if done). If the goal is not yet achieved, the agent loops back to the Reason step. This loop continues until the agent determines it has enough information to provide a final response.

## Where AI Agent Canvas Fits

AI Agent Canvas is a multi-agent enterprise copilot framework built on three pillars:

- **Microsoft Agent Framework (MAF)** for agent orchestration and the LLM execution loop
- **Azure AI Foundry** as the LLM provider (GPT-4o, GPT-4.1, and other models)

- **CopilotKit with the AG-UI protocol** for real-time streaming to the frontend

The framework is designed around separation of concerns: Core handles the agent execution loop, tool registry, context providers, and streaming. MCP projects provide data connections and external tool integrations. Custom agents define personas, domain logic, and specialized behavior. The Web project is the composition root that wires everything together.

# Microsoft Agent Framework

Microsoft Agent Framework (MAF) is the orchestration layer that powers AI Agent Canvas. It provides the abstractions for building agents that can reason, call tools, and stream responses -- all while integrating with the broader Microsoft.Extensions.AI ecosystem. AI Agent Canvas uses MAF version 1.10.0 from the Microsoft.Agents.AI namespace.

## Core Types

**IChatClient** -- At the foundation, MAF builds on IChatClient from Microsoft.Extensions.AI. This interface represents any LLM provider -- Azure AI Foundry, OpenAI, Ollama, or others. AI Agent Canvas configures it through AIFoundryClientFactory, meaning the framework is not locked to a specific LLM provider.

**ChatClientAgent** -- The primary agent implementation. It combines an IChatClient with configuration (tools, context providers, history) and implements the agent loop: receive messages, inject context, call LLM, execute tool calls, repeat until done.

**ChatClientAgentOptions** -- Configuration class that defines agent behavior including Name, Description, ChatOptions (with tool list), ChatHistoryProvider, and AIContextProviders chain.

**AIAgent** -- Abstract base class for all agents in MAF. ChatClientAgent inherits from it. The AIAgent type is what gets registered in DI and consumed by the AG-UI endpoint.

## The Agent Execution Model

When a user sends a message, the following pipeline executes: (1) User message arrives at the AG-UI endpoint. (2) Context providers inject system prompt, persona, guardrails, entity knowledge, RAG results, and governance checks. (3) Messages are assembled with injected context. (4) Tool definitions are attached from the tool registry. (5) The LLM is invoked. (6) If the LLM returns tool calls, the framework executes them and loops back to step 5 with the results. (7) When the LLM returns text instead of tool calls, the response streams back via SSE.

## Middleware Pipeline

MAF supports a middleware pattern for cross-cutting concerns. AI Agent Canvas uses this to add logging middleware that records agent invocation timing, message counts, and completion status. The middleware wraps the chat client, intercepting calls to add observability without modifying agent logic.

## Running the Agent

MAF provides two execution methods: **RunAsync** for complete responses (used by scheduled tasks and background jobs) and **RunStreamingAsync** for SSE streaming (used by the AG-UI endpoint for real-time chat). Both methods participate in the same tool-calling loop; the difference is whether the final text response is returned as a complete string or streamed token-by-token.

# Tools and Skills

Tools are how agents interact with the outside world. When the LLM determines it needs external data or wants to perform an action, it requests a tool call. The framework executes the tool and returns the result to the LLM for further reasoning.

## AIFunction and AIFunctionFactory

Tools are created using AIFunctionFactory.Create(), which wraps a .NET method as an AITool with a name, description, and parameter schema. The LLM uses the description and schema to decide when and how to call the tool.

```
AIFunctionFactory.Create(GetStockQuote, "stock_quote", "Get real-time stock quote including price, volume, and change")
```

## The Tool Call Loop

When the LLM decides to use a tool: (1) The LLM returns a tool_call with the function name and arguments. (2) The framework looks up the tool in the registry and executes it. (3) The tool result is added to the conversation as a tool response message. (4) The LLM is called again with the updated conversation, allowing it to reason about the result and decide whether to call another tool or generate a final response.

## DynamicToolRegistry

A thread-safe registry (ConcurrentDictionary) that enables runtime tool registration and unregistration. Tools are grouped by source key (e.g., 'mcp:weather-api'). This powers MCP connections -- when an agent connects to an MCP server at runtime, the discovered tools are registered under a source key. Disconnecting removes them.

## Tool Providers

AI Agent Canvas ships with 70+ registered tools across multiple providers:

| Provider | Tools |
|---|---|
| PersonaToolProvider | create/update/list/switch/read/delete_persona |
| GuardrailToolProvider | create/update/list/toggle/delete_guardrail |
| WorkflowToolProvider | create/list/read/run/delete_workflow |
| EntityToolProvider | save/update/read/search/list/delete_entity |
| GoalToolProvider | create_goal, list_goals, read_goal, update_goal_status, delete_goal |
| WorkQueueToolProvider | submit_work_item, list_work_queue, cancel_work_item, get_queue_stats |

| Provider | Tools |
|---|---|
| MarketDataToolProvider | edgar_company_facts, stock_quote, stock_history |
| SkillToolProvider | create_skill, list_skills, run_skill, remove_skill |
| McpConnectionManager | connect_mcp_server, disconnect_mcp_server, list_mcp_connections |
| SchedulerToolProvider | schedule_recurring_task, schedule_one_time_task, list/remove/get_results |
| SchedulerToolProvider (autonomous) | start_autonomous_mode, stop_autonomous_mode, get_autonomous_status |
| AgentRegistryToolProvider | list_available_agents, get_agent_info |
| AgentMailboxToolProvider | send_to_agent, check_inbox, reply_to_message |
| HandoffToolProvider | handoff_to_agent |
| SystemToolsProvider | system_read_file, system_write_file, system_list_directory, system_run_script |

## Skills: Agent-Managed Tools

Skills are reusable prompt templates that the agent can create, store, and execute at runtime. Unlike hardcoded tools, skills are authored through conversation and persisted as markdown files in agent-data/skills/. Each skill has a name, description, and prompt template with an {input} placeholder. When run, the template is populated with the user's input and sent to the LLM.

# Model Context Protocol

MCP is an open standard for connecting AI agents to external data sources and tools. Instead of building custom integrations for every data source, MCP provides a standardized protocol for tool discovery and invocation.

## Why MCP Matters

Without MCP, every data connection requires custom code: HTTP client setup, response parsing, error handling, and tool registration. MCP standardizes this -- an agent connects to any MCP-compliant server and automatically discovers its available tools, schemas, and capabilities.

## MCP in AI Agent Canvas

AI Agent Canvas supports both static MCP connections (configured at startup via IMcpConnectionSeed) and dynamic runtime connections (the agent calls connect_mcp_server during a conversation). McpConnectionManager handles the lifecycle: creating transports (HTTP or SSE), connecting clients, discovering tools via ListToolsAsync, and registering them in the DynamicToolRegistry.

## MCP Tools vs Local Tools

| Aspect | Local Tools | MCP Tools |
|---|---|---|
| Definition | Code in the project | External MCP server |
| Registration | Startup (DI) | Runtime (dynamic) |
| Execution | In-process | Network call |
| Latency | Microseconds | Milliseconds |
| Availability | Always | While connected |

## Security Considerations

MCP connections are subject to governance policies via GovernedMcpGateway. This prevents SSRF attacks by blocking connections to private/internal addresses, enforces tool-level access control, and logs all MCP activity for audit. Best practices include endpoint allowlisting, HTTPS enforcement, and timeout configuration.

# Context Providers

Context providers are the mechanism for injecting instructions, tools, and knowledge into each LLM call. They form a chain that executes before every agent invocation, assembling the complete context the LLM needs to respond appropriately.

## The Context Injection Chain

Eight built-in context providers plus one middleware execute in sequence:

| Component | Purpose |
|---|---|
| SystemPromptProvider | Sets base system instructions |
| PlanningMiddleware | Goal decomposition: persistent step plans via StateBag |
| PersonaContextProvider | Appends persona-specific behavior instructions |
| UserProfileContextProvider | Injects user preferences and information |
| EntityContextProvider | Appends domain knowledge index |
| GuardrailContextProvider | Appends behavioral constraints and safety rules |
| RagContextProvider | Retrieves relevant documents from vector store |
| GovernanceContextProvider | Scans for prompt injection, emits audit events |
| DynamicToolContextProvider | Injects runtime tools from DynamicToolRegistry |

## Goal Decomposition (PlanningMiddleware)

The PlanningMiddleware enables persistent goal decomposition for complex multi-step requests. Implemented as agent middleware (not a context provider), it has access to the session StateBag for plan persistence across messages. On each message, the middleware checks for an existing plan: if none, it makes a lightweight LLM call (MaxOutputTokens=300, Temperature=0) to decide whether a plan is needed. Simple requests get NO_PLAN and pass through unchanged. Complex requests are decomposed into numbered steps referencing available tools. The plan is stored in StateBag and injected as a system message. On subsequent messages, a continuation evaluator checks conversation history to track which steps are complete (COMPLETED/REMAINING/NEXT), clears completed plans (ALL_DONE), or generates a fresh plan if the user changed direction (REPLAN). The planner is aware of both static startup tools and dynamic MCP tools via DynamicToolRegistry.

## Creating Custom Context Providers

Custom providers subclass AIContextProvider and override ProvideAIContextAsync. Register them in DI to add domain-specific context injection. Design guidelines: single responsibility per provider, append (don't replace)

existing context, fail gracefully, keep instructions concise, and register in logical order.

# AG-UI Protocol

AG-UI is the streaming protocol that connects the .NET agent backend (Microsoft Agent Framework) to the CopilotKit frontend over Server-Sent Events (SSE). Instead of returning a complete response in a single payload, AG-UI streams events as the agent processes, enabling token-by-token text rendering.

## The /api/copilotkit Endpoint

A single SSE endpoint handles all agent interactions. CopilotKit sends a POST request with a JSON body containing threadId and messages array. The endpoint responds with text/event-stream content type.

## Event Types

| Event | Purpose |
|---|---|
| run.started | Marks agent processing initiation; frontend shows loading indicator |
| text.message.start | New assistant message; frontend creates message bubble |
| text.message.content | Text chunk with delta field; frontend appends progressively |
| text.message.end | Message complete |
| run.finished | Entire run complete; frontend re-enables input |

## Why SSE Over WebSockets

SSE is unidirectional (server to client), which matches the agent response pattern perfectly. It works over standard HTTP infrastructure (proxies, CDNs, load balancers) without modification, provides automatic reconnection, and avoids the complexity of WebSocket connection management and heartbeats.

The endpoint respects the HTTP request's cancellation token -- if the client closes the connection, the RequestAborted token fires, canceling the agent's LLM call and stopping the stream to prevent wasted compute.

# Notification System

AI Agent Canvas includes a real-time notification system that delivers scheduled task results and other agent events to the chat interface. Notifications flow through an in-memory channel-based pub/sub (InMemoryNotificationSink) and are delivered to the frontend via a dedicated SSE endpoint at /api/notifications.

## How It Works

When a scheduled task completes, ScheduledAgentJob publishes an AgentNotification to the INotificationSink. The NotificationEndpoint reads from this channel and streams each notification as an SSE event with the 'notification' event type. The frontend listens with EventSource and renders each notification as a new assistant message in the chat.

## Architecture

| Component | Role |
| --- | --- |
| INotificationSink | Abstraction for publishing notifications (in Abstractions) |
| InMemoryNotificationSink | Channel-based implementation using Channel<AgentNotification> |
| NotificationEndpoint | SSE endpoint at /api/notifications that streams to browser |
| AgentNotification | DTO with Title, Body, Source, and Timestamp fields |
| EventSource (frontend) | Browser API that receives SSE events and updates chat state |

# Use Cases

Real-world scenarios showing how AI Agent Canvas applies to financial services, healthcare, legal, e-commerce, and IT operations.

# Use Cases

AI Agent Canvas is a multi-agent enterprise copilot framework that adapts to any industry. The following scenarios demonstrate how organizations use tool-calling agents, MCP data connections, scheduled tasks, skill authoring, personas, guardrails, entity management, user profiles, and security governance to solve real operational challenges. Each use case presents a hypothetical company, a specific persona, the problem they face, and how AI Agent Canvas delivers measurable value.

# Financial Services: Market Intelligence Copilot

**Company:** Meridian Capital -- A mid-size hedge fund ($2.4B AUM) giving junior analysts instant access to market data, SEC filings, and technical indicators while enforcing compliance guardrails.

**Persona:** Priya Sharma, Junior Portfolio Analyst

## The Challenge

Priya spends 2-3 hours each morning manually pulling data from Bloomberg, SEC EDGAR, and internal spreadsheets to prepare briefing materials. She needs real-time market data combined with fundamental analysis, but compliance requires that all outputs include proper disclaimers and avoid direct trading recommendations.

## Key Features Demonstrated

- **MCP.MarketData tools:** stock_quote, edgar_company_facts, stock_technicals for real-time market data and SEC filings
- **Scheduled tasks:** Morning Watchlist Briefing (6:00 AM daily), Earnings Calendar Alert (weekly Monday)
- **Personas:** Research Mode (verbose analysis with citations) and Briefing Mode (concise executive bullets)
- **Guardrails:** NoTradingSignals (blocks buy/sell recommendations), SourceAttribution (requires data citations)
- **Skills:** Custom prompt templates for earnings analysis and sector comparison

## Sample Interaction

- "What is the current price and 30-day performance of CAT?"
- "Pull CAT's latest 10-K revenue breakdown by segment"
- "Show me the RSI and MACD for CAT over the last 90 days"
- "Create a skill for comparing two stocks side by side"

## Business Value

Morning prep time reduced from 2-3 hours to 15 minutes. Automated watchlist briefings deliver pre-market intelligence before the trading day starts. Compliance guardrails ensure all analyst outputs include required disclaimers.

# Healthcare: Clinical Research Assistant

**Company:** BioNova Therapeutics -- A 90-person biotech startup managing multi-site Phase II clinical trials for three drug candidates.

**Persona:** Dr. Marcus Chen, Clinical Research Coordinator

## The Challenge

Dr. Chen coordinates enrollment, adverse events, and endpoint data across 12 trial sites. Data lives in multiple systems, and he needs quick answers about enrollment velocity, overdue visits, and safety signals without waiting for the data team to run custom queries.

## Key Features Demonstrated

- **MCP.ClinicalTrials tools:** query_enrollment, query_adverse_events, query_endpoints
- **Entity Management:** Drug compound registry (BNV-204, BNV-118, BNV-330) with trial phase, mechanism, and status
- **Personas:** Research Mode (exploratory, allows speculation) and Regulatory Mode (strict, citation-required)
- **Guardrails:** PatientPrivacy (blocks PII), NoDiagnosticClaims (prevents medical conclusions)
- **Scheduled tasks:** Weekly enrollment dashboard (Monday 6 AM), overdue visit alerts (daily 8 AM)

## Business Value

Enrollment queries that took hours via the data team now complete in seconds. Automated overdue visit detection catches protocol deviations within 24 hours instead of weekly batch reports. Privacy guardrails ensure no patient-identifiable information appears in chat responses.

# Legal: Contract Review Agent

**Company:** Sterling and Associates -- A 120-attorney law firm in Chicago specializing in technology transactions and IP licensing.

**Persona:** Elena Vasquez, Associate Attorney

## The Challenge

Elena reviews 15-20 technology contracts monthly, each 50-100 pages. She needs to identify non-standard clauses, compare against firm precedents, and check client-specific requirements. Manual review takes 4-6 hours per contract.

## Key Features Demonstrated

- **MCP.DocumentManagement tools:** search_precedents, retrieve_clause, check_client_standards
- **Custom skills:** ClauseExtraction (extract IP assignment clauses), RedlineComparison, PositionReconciliation
- **Personas:** Associate Mode (detailed analysis with full citations) and Partner Mode (concise executive summary)
- **Guardrails:** MatterBasedAccessControl, NoLegalAdviceFraming, PrivilegeProtection
- **User Profiles:** Practice area, seniority level, matter assignments for personalized access

## Business Value

Contract review time reduced from 4-6 hours to 90 minutes. Clause extraction skills ensure consistent identification of non-standard terms. Matter-based access controls prevent cross-client data leakage.

# E-Commerce: Customer Operations Copilot

**Company:** Bloom and Vine -- A D2C home goods company in Portland, OR ($18M annual revenue) with a 3-person customer operations team.

**Persona:** Aisha Thompson, Customer Operations Manager

## The Challenge

Aisha's team handles 200+ support tickets weekly across Zendesk, Shopify, and email. Returns processing requires checking order status, verifying return eligibility, creating return labels, and drafting customer responses -- a 15-minute workflow per ticket.

## Key Features Demonstrated

- **MCP.Shopify tools:** lookup_order, lookup_customer, process_return, check_inventory
- **MCP.Zendesk tools:** get_ticket, update_ticket, search_tickets, create_macro
- **Scheduled tasks:** Order Anomaly Scan (2-hourly), Shipping Delay Monitor (4-hourly), Weekly Ticket Trends
- **Workflows:** ReturnsProcessing (policy check then return creation then label then response draft)
- **Guardrails:** RequireConfirmation (for destructive actions), SpendLimit ($200 threshold for escalation)

## Business Value

Returns processing drops from 15 minutes to 3 minutes per ticket. Automated anomaly scans catch shipping delays before customers complain. Weekly trend reports identify recurring product issues.

# IT Operations: Incident Response Agent

**Company:** CloudScale Systems -- A B2B SaaS platform with 55 engineers and a 6-person SRE team managing 300+ microservices.

**Persona:** Jordan Park, Site Reliability Engineer

## The Challenge

Jordan handles 3-5 incidents per week, each requiring correlation across Datadog metrics, deployment history, and service dependency graphs. Mean time to root cause is 45 minutes, with most time spent context-switching between dashboards.

## Key Features Demonstrated

- **MCP.Observability tools:** query_metrics, get_alerts, get_deployment_history, query_logs, get_service_dependencies
- **Scheduled health checks:** SLA Budget (15 min), Deployment Correlation (30 min), Dependency Health (hourly), Certificate Expiry (daily)
- **Personas:** Incident Mode (terse, action-oriented with escalation thresholds) and Analysis Mode (detailed, trend-focused)
- **Guardrails:** ReadOnlyDefault, DestructiveActionBlocklist, BlastRadiusAssessment, IncidentEscalationRules
- **Entity Management:** Service catalog with tier, owner, SLA target, dependencies, and runbook links

## Business Value

Mean time to root cause drops from 45 to 12 minutes. Automated deployment correlation catches bad deploys within the SLA budget check interval. Service catalog entities provide instant context on service ownership and runbooks during incidents.

# User Guide

Getting started, configuration, chat interface, personas, skills, scheduling, workflows, and security.

# Getting Started

## Prerequisites

- .NET 9 SDK
- Node.js 18+ and npm
- Azure AI Foundry account with a deployed model (GPT-4o or GPT-4.1)

## Quick Start

```
git clone &lt;your-repo-url&gt; AiAgentCanvas cd AiAgentCanvas dotnet build AiAgentCanvas.sln
# Configure appsettings.json with your Azure AI Foundry credentials cd src/AiAgentCanvas.Web
dotnet run # In another terminal: cd frontend npm install npm run dev
```

Open the chat interface at **http://localhost:5149** (backend serves static frontend) or **http://localhost:3000** (Next.js dev server with API proxy).

## Your First Conversation

- "What can you help me with?" -- See available capabilities
- "Get me a stock quote for AAPL" -- Test market data tools
- "Create a persona called Analyst focused on data-driven insights" -- Create a persona
- "Schedule a recurring task every hour to check the S&P 500 price" -- Test scheduling

# Configuration

Configuration is managed through appsettings.json in the AiAgentCanvas.Web project.

## AIFoundry Section

```
{ "AIFoundry": { "Endpoint": "https://your-resource.openai.azure.com/", "Key":
"your-api-key-here", "DeploymentName": "gpt-4o", "UseAzureCredential": false } }
```

Set **UseAzureCredential** to true for Azure Managed Identity (recommended for production). When enabled, the Key field is ignored and DefaultAzureCredential handles authentication.

## Security Section

```
{ "Security": { "PolicyPath": "governance-policy.yaml", "RateLimitPerMinute": 30 } }
```

## Environment Variables

All configuration values can be overridden with environment variables using double-underscore notation:

```
AIFoundry__Key=your-key-here Security__RateLimitPerMinute=60
```

## Runtime Data Directories

Agent data is stored in markdown files under **agent-data/** with subdirectories for each domain: personas, context, workflows, guardrails, entities, profiles, goals, and skills. SQLite databases are also created at runtime: workqueue.db (autonomous execution work queue), agentmailbox.db (inter-agent messages), hangfire.db (scheduled tasks), skills.db, chathistory.db, and optionally vectorstore.db.

# Chat Interface

The chat interface provides a streaming conversational experience powered by the AG-UI protocol.

**Sending Messages:** Type a message and press Enter or click Send. The agent processes your message through the full pipeline: context injection, LLM reasoning, tool calls (if needed), and response streaming.

**Streaming Responses:** Responses appear token-by-token as the agent generates them. Tool calls execute in the background -- you will see the agent reason about results and continue generating.

**Notifications:** Scheduled tasks publish notifications that appear as chat messages via the SSE notification channel at /api/notifications. When a background task completes, its result is delivered via Server-Sent Events and displayed inline in the conversation.

**Conversation Context:** The agent maintains conversation history within a session. Reference earlier messages naturally -- the agent has full context of the conversation.

# Personas

Personas define how the agent behaves -- its tone, expertise, focus areas, and communication style. They are injected as system instructions before each LLM call.

## Managing Personas

- "Create a persona called Financial Analyst with instructions to focus on data-driven insights and risk assessment"
- "List my personas" -- See all available personas
- "Switch to the Financial Analyst persona" -- Activate a persona
- "Show me the Financial Analyst persona" -- View full details
- "Update the Financial Analyst persona to also include ESG analysis"
- "Delete the Financial Analyst persona"

Personas are stored as markdown files in **agent-data/personas/** with YAML frontmatter containing name and description, and a body containing the system instructions. The active persona is tracked in **agent-data/personas/.active**.

# Workflows

Workflows are multi-step procedures stored as markdown files. They define a sequence of instructions the agent follows when the workflow is run.

## Managing Workflows

- "Create a workflow called Morning Briefing that gets stock quotes for AAPL, MSFT, and GOOGL, then summarizes market sentiment"
- "List my workflows"
- "Run the Morning Briefing workflow"
- "Delete the Morning Briefing workflow"

Workflows are stored in **agent-data/workflows/** as markdown files with YAML frontmatter (name, description, tags) and step-by-step instructions in the body. They can be combined with scheduling for automated recurring analysis.

# Skills and Tools

Skills are reusable prompt templates that you create through conversation. They are persisted as markdown files and can be run with any input.

## Creating and Running Skills

- "Create a skill called summarize-earnings with description Summarize quarterly earnings and template Analyze the following earnings report and provide key highlights: {input}"
- "List my skills" -- See all saved skills
- "Run the summarize-earnings skill with input: [paste earnings data]"
- "Remove the summarize-earnings skill"

## Skill Authoring (Markdown-Based)

Skills can also be authored as markdown files with YAML frontmatter for richer definitions. The SkillAuthoringToolProvider exposes tools for creating, editing, reading, and deleting authored skills directly in the agent-data/skills/ directory.

## Connecting External MCP Servers

- "Connect to an MCP server at https://weather-api.example.com with transport http"
- "List my MCP connections" -- See active connections and tool counts
- "Disconnect from the weather server"

# Scheduling and Autonomous Execution

Scheduled tasks run the AI agent with a given prompt on a cron schedule or after a delay. Tasks are managed by Hangfire with SQLite persistence and survive application restarts. Beyond scheduling, AI Agent Canvas also supports fully autonomous execution where the agent works independently on goals.

## Creating Tasks

- "Schedule a recurring task every day at 9 AM to check the S&P 500 price" (cron: 0 9 * * *)
- "Schedule a one-time task in 30 minutes to analyze AAPL earnings"

## Common Cron Expressions

| Schedule | Cron Expression |
|---|---|
| Every hour | 0 * * * * |
| Every day at 9 AM | 0 9 * * * |
| Every Monday at 8 AM | 0 8 * * 1 |
| Every 30 minutes | */30 * * * * |
| Every weekday at 6 PM | 0 18 * * 1-5 |

## Managing Tasks

- "List my scheduled tasks" -- See all active tasks
- "Get task results" -- View recent completed task outputs
- "Remove scheduled task [task-id]" -- Cancel a task

The Hangfire Dashboard is available at **/hangfire** for visual monitoring of jobs.

When a scheduled task completes, it publishes a notification via the SSE notification channel. If the chat interface is open, the result appears as a new assistant message in the conversation.

## Autonomous Execution

Enable autonomous mode with: *"Start autonomous mode"*. The agent will work independently on goals, polling the work queue and executing items via AIAgent.RunAsync. Create goals with priority and acceptance criteria, submit work items directly, or let the autonomous job decompose goals into work items automatically. Monitor with *"Get autonomous status"* and stop with *"Stop autonomous mode"*. See the Developer Guide for full configuration details.

# Security

AI Agent Canvas integrates Microsoft's Agent Governance Toolkit for comprehensive security controls.

## Governance Policy

Policies are defined in YAML files that specify allow/deny rules for tools, file paths, and MCP endpoints.

```
name: AiAgentCanvas-default rules: - name: block-dangerous-tools action: deny tools: [run_script] - name: restrict-file-write-paths action: deny tools: [write_file] paths: [/etc, /var, C:\Windows] - name: allow-all-other action: allow
```

## Security Features

- **Rate Limiting:** Fixed-window rate limiter (default 30/minute) on the copilotkit endpoint
- **Prompt Injection Detection:** GovernanceContextProvider scans instructions before each LLM call
- **Tool-Call Governance:** Policies evaluated before tool execution (allow/deny/audit)
- **MCP Gateway:** GovernedMcpGateway prevents SSRF and enforces endpoint allowlists
- **Security Headers:** X-Content-Type-Options, X-Frame-Options, Referrer-Policy
- **Audit Logging:** All governance decisions are logged for compliance review

## Guardrails (User-Defined Safety Rules)

Guardrails are runtime-configurable behavioral constraints injected into the system prompt. Create them through conversation:

- "Create a guardrail called no-trading-signals with rule: Never provide buy, sell, or hold recommendations"
- "Toggle the no-trading-signals guardrail" -- Enable/disable
- "List my guardrails" -- See all active rules

## Production Security Checklist

- Use Managed Identity instead of API keys
- Customize governance-policy.yaml for your domain
- Add authentication to the Hangfire dashboard
- Enable HTTPS/TLS
- Set AllowedHosts to your specific domain
- Review audit logs regularly

# Developer Guide

Architecture, project structure, core framework internals, agent data domains, skills, MCP, security, and custom agent development.

# Architecture Overview

AI Agent Canvas is a .NET 9 multi-agent copilot framework built on Microsoft's Agent Framework (MAF). It connects a Next.js frontend to Azure AI Foundry through the AG-UI streaming protocol, with a modular tool registry and governance layer.

## Request Flow

Browser User sends message via CopilotKit Frontend, which POSTs to /api/copilotkit. The AgUiEndpoint receives the request and passes it to ChatClientAgent. Context Providers inject system prompt, persona, guardrails, entity knowledge, RAG results, and governance checks. The agent calls Azure AI Foundry LLM, executes tool calls in a loop, and streams the response back via SSE to the Frontend UI.

## Framework vs Custom Separation

The solution enforces a strict boundary: framework projects (src/AiAgentCanvas.*) handle orchestration, tools, and persistence; custom projects (src/Custom/*) contain business-specific agents and data connections. Custom projects never reference framework internals -- they register tools and prompts through DI, and the framework wires everything together.

## Key Packages

| Package | Version | Purpose |
|---|---|---|
| Microsoft.Agents.AI | 1.10.0 | Agent orchestration (ChatClientAgent, AIAgent) |
| Microsoft.Extensions.AI | 10.7.0 | IChatClient, AITool, embeddings abstraction |
| ModelContextProtocol | 1.4.0 | MCP client for external tool connections |
| AgentGovernance | Latest | Policy engine, prompt injection, audit |
| Azure.AI.OpenAI | 2.2.0 | Azure AI Foundry client |
| Hangfire | 1.8+ | Background job scheduling |

## Extension Method Pattern

Every module follows the same DI registration pattern: an AddAiAgentCanvas*() extension method on IServiceCollection that registers stores, tool providers, context providers, and tools as IReadOnlyList<AITool> singletons. The composition root (Program.cs) calls these methods to assemble the application.

# Project Structure

## Solution Layout

| Project | Purpose |
|---|---|
| AiAgentCanvas.Abstractions | Shared interfaces, seed contracts, MarkdownFile, DocumentRecord |
| AiAgentCanvas.Core | Agent orchestration, AG-UI endpoint, tool registry, context providers, inter-agent communic |
| AiAgentCanvas.AgentData | Seven data domains: personas, context, workflows, guardrails, entities, profiles, goals + w |
| AiAgentCanvas.Skills | Skill store, MCP connections, skill registry, skill authoring |
| AiAgentCanvas.Scheduler | Hangfire job scheduling, task store, autonomous execution engine |
| AiAgentCanvas.Notifications | In-memory notification sink with SSE endpoint |
| AiAgentCanvas.Security | Agent Governance integration, rate limiting, security headers |
| AiAgentCanvas.SystemTools | Sandboxed file I/O and shell execution |
| AiAgentCanvas.Web | Composition root (Program.cs), static frontend hosting |
| Custom/HelloWorldAgent | Example agent with full seed implementation |
| Custom/MCP.MarketData | Yahoo Finance and SEC EDGAR tool providers |
| Custom/VectorStore.Sqlite | SQLite-backed vector store for RAG embeddings |

## Dependency Flow

All projects depend downward to Abstractions. The Web project references everything. Custom projects reference only Abstractions and Microsoft.Extensions.AI -- never framework internals. This ensures custom agents remain decoupled from the framework implementation.

# Core Framework

The Core project is the central orchestration engine.

## ServiceCollectionExtensions.AddAiAgentCanvas()

Registers: AiAgentCanvasOptions, AIFoundryClientFactory, DynamicToolRegistry, all context providers, ChatClientAgent (via AIAgentBuilder), CORS policy, and tool dependency validation.

## Tool Registration: Two Paths

**Startup registration:** Modules register IReadOnlyList<AITool> as singleton services. Core collects all of these and passes them to ChatClientAgentOptions.ChatOptions.Tools.

**Runtime registration:** DynamicToolRegistry allows tools to be added/removed at runtime. DynamicToolContextProvider merges these into ChatOptions before each LLM call.

## Tool Dependency Validation

At startup, Core resolves all IToolDependencySeed services and validates that the required tools are registered. Missing tools are logged as warnings, allowing the application to start while surfacing configuration issues.

```
foreach (var dep in sp.GetServices&lt;IToolDependencySeed&gt;()) { var missing =
dep.RequiredTools .Where(t =&gt; !toolNames.Contains(t)).ToList(); if (missing.Count &gt; 0)
logger.LogWarning( "Agent '{Agent}' requires tools: {Missing}", dep.AgentName, string.Join(",
", missing)); }
```

# Agent Data

Agent data is organized into seven domains, each following the same three-class pattern: Store (CRUD on markdown files), ToolProvider (LLM-callable tools), and ContextProvider (injection before each LLM call). The Goals domain additionally includes a SQLite-backed work queue for autonomous execution.

## The Seven Domains

| Domain | Storage Path | Context Injection |
|---|---|---|
| Personas | agent-data/personas/ | Active persona instructions appended to system prompt |
| Guardrails | agent-data/guardrails/ | Enabled rules appended as behavioral constraints |
| Workflows | agent-data/workflows/ | Available as tools (no context injection) |
| Context | agent-data/context/ | All context documents appended as knowledge |
| Entities | agent-data/entities/ | Entity index appended as domain knowledge |
| User Profiles | agent-data/profiles/ | Active profile preferences injected for personalization |
| Goals | agent-data/goals/ | Managed via tools; drives autonomous execution job |

## MarkdownFile: The Persistence Foundation

All agent data uses the same file format: YAML frontmatter (delimited by ---) containing metadata, followed by markdown body content. MarkdownFile provides Parse(), Write(), LoadAll(), and SanitizeFileName() methods. This approach makes all agent data human-readable, version-controllable, and editable outside the application.

```
--- name: financial-analyst description: Data-driven market analysis persona --- You are a financial analyst. Focus on quantitative data, risk assessment, and actionable insights.
```

## Goals Domain

The Goals domain provides markdown-persisted goal definitions and a SQLite-backed transient work queue. Goals have name, description, priority (critical/high/medium/low), status, acceptance criteria, and optional assigned agent. The GoalStore follows the same agent/user directory split as other domains. The WorkQueueStore (workqueue.db) manages individual work items with priority-ordered claiming via atomic SQL.

## Creating a New Domain

To add a new domain: (1) Create a subdirectory in agent-data/. (2) Create a Store class with CRUD methods using MarkdownFile. (3) Create a ToolProvider with AIFunctionFactory tools. (4) Optionally create a ContextProvider. (5) Create an AddAiAgentCanvas*() extension method. (6) Register in Program.cs.

# Skills and MCP

The Skills project provides four components: SkillStore (CRUD for prompt templates), McpConnectionManager (runtime MCP connections), LocalSkillRegistry (skill catalog), and SkillAuthoringToolProvider (markdown-based skill creation).

## SkillStore

Skills are persisted as markdown files in agent-data/skills/ using the same MarkdownFile format as all other agent data domains. Each skill has a name, description, and prompt template with {input} placeholder in the markdown body.

## McpConnectionManager

Manages runtime MCP connections. When the agent calls connect_mcp_server: (1) Creates an HttpClientTransport or SseClientTransport. (2) Connects via McpClient. (3) Discovers tools via ListToolsAsync. (4) Registers tools in DynamicToolRegistry under 'mcp:name' key. Disconnecting removes the tools and disposes the client.

## IMcpConnectionSeed: Static Connections

For MCP connections that should be established at application startup (rather than dynamically by the agent), implement IMcpConnectionSeed. This interface declares Name, Endpoint, and Transport. At startup, the framework resolves all IMcpConnectionSeed instances and connects to them automatically, registering their tools in the DynamicToolRegistry.

## MCP Connection Lifecycle

MCP connections are subject to governance policies via GovernedMcpGateway. The gateway can block connections to private/internal addresses (SSRF prevention), require tool approval, and log all activity for audit.

# RAG Pipeline

The RAG (Retrieval-Augmented Generation) pipeline enriches agent responses with relevant documents from a vector store. It goes beyond basic vector search with recursive chunking, hybrid search, metadata filtering, LLM-based reranking, and citation attribution.

## Pipeline Stages

| Stage | Component | Description |
|---|---|---|
| 1. Chunking | DocumentChunker | Recursive split: paragraphs then sentences, with overlap |
| 2. Embedding | IEmbeddingGenerator | Azure AI Foundry text-embedding model |
| 3. Storage | SqliteDocumentCollection | Vector BLOB + FTS5 full-text index |
| 4. Hybrid Search | IHybridSearchable | 70% cosine similarity + 30% BM25 keyword |
| 5. Filtering | RagSearchOptions | Source (exact) and Tags (contains) SQL filters |
| 6. Reranking | LlmReranker | LLM rescores top-10 candidates, keeps top-3 |
| 7. Citation | RagContextProvider | Numbered citations with source and score |

## Document Chunking

The DocumentChunker in Core recursively splits documents into chunks suitable for embedding. It splits by paragraphs first (double newlines), then by sentences if a paragraph exceeds ChunkSize (default 512 chars). Configurable overlap (default 64 chars) provides context continuity between consecutive chunks. Chunks under 20 characters are discarded.

## Hybrid Search

The SQLite vector store maintains both a main table (with embedding BLOBs) and an FTS5 full-text index. At query time, both cosine similarity and BM25 keyword scores are computed and combined: finalScore = (vectorWeight * cosineSimilarity) + (keywordWeight * bm25Score). Default weights are 70% vector, 30% keyword. The IHybridSearchable interface in Abstractions keeps Core decoupled from the SQLite implementation.

## Metadata Filtering

DocumentRecord includes Source (e.g., 'annual-report-2024.pdf') and Tags (e.g., 'finance,quarterly') fields. RagSearchOptions supports SourceFilter (exact match) and TagFilter (LIKE match) applied as SQL WHERE clauses before vector scoring, reducing the candidate set.

## LLM-Based Reranking

After retrieval, LlmReranker sends the query and top-10 candidates (truncated to 300 chars each) to the LLM, asking it to return a JSON array ranking them by relevance. The top-3 from the reranked list are kept. This cross-encoder pattern produces better relevance judgments than embedding similarity alone. Uses MaxOutputTokens=100 and Temperature=0 for speed and determinism. Falls back to original ranking on any error.

## Citation and Attribution

RagContextProvider formats each result as a numbered citation: [1] (source: filename.pdf, score: 0.892) followed by a text preview. The LLM is instructed to 'cite by number when using' these documents, producing responses like: 'Revenue grew 12% to $4.2B [1], with management focusing on AI infrastructure [2].'

## Architecture

| Component | Location | Purpose |
|---|---|---|
| DocumentRecord | Abstractions | DTO with Id, Text, Source, Tags, Embedding |
| DocumentChunk | Abstractions | Chunker output: Text, Index, Source |
| RagSearchOptions | Abstractions | Filters and weights for hybrid search |
| IHybridSearchable | Abstractions | Interface for hybrid vector + keyword search |
| DocumentChunker | Core | Recursive paragraph/sentence text splitter |
| LlmReranker | Core | LLM-based candidate reranking |
| RagContextProvider | Core | Orchestrates search, rerank, citation, injection |
| SqliteDocumentCollection | VectorStore.Sqlite | SQLite + FTS5 hybrid search implementation |

# Inter-Agent Communication

AI Agent Canvas supports multi-agent collaboration through three mechanisms registered via AddAiAgentCanvasInterAgentCommunication() in Core.

## Agent Registry

AgentRegistry builds and caches named ChatClientAgent instances from persona definitions. Each persona becomes a resolvable agent with its own instructions, sharing the same tools and context providers as the main agent. The registry uses a ConcurrentDictionary cache and delegate-based persona lookup to avoid circular project dependencies (Core cannot reference AgentData).

## Agent Mailbox

AgentMailbox is a SQLite-backed per-agent message queue (agentmailbox.db) for asynchronous communication. Messages have sender, recipient, content, status (pending/read/replied), and optional response. Tools: send_to_agent, check_inbox, reply_to_message.

## Handoff (Synchronous Delegation)

The handoff_to_agent tool runs a target agent synchronously using AIAgent.RunAsync and returns the result to the calling agent in the same turn. This enables one agent to delegate a subtask to a specialist agent and reason about its response before replying to the user.

## Inter-Agent Tools

| Tool | Description |
|---|---|
| list_available_agents | List all agents available for handoff or messaging |
| get_agent_info | Get details about a specific agent (persona name, description) |
| handoff_to_agent | Synchronous delegation: run target agent and return result |
| send_to_agent | Send async message to another agent's mailbox |
| check_inbox | Check for pending messages from other agents |
| reply_to_message | Reply to a message from another agent |

# Autonomous Execution

The Scheduler project includes an autonomous execution engine that allows the agent to work independently on goals without user input.

## How It Works

When autonomous mode is enabled, a Hangfire recurring job (AutonomousAgentJob) runs on a configurable schedule. Each run: (1) Polls WorkQueueStore.ClaimNext() for pending work items, prioritized by urgency. (2) If no work items exist, picks the next active goal (sorted by priority) that doesn't already have pending work items. (3) Creates a work item from the goal and claims it. (4) Executes the work item via AIAgent.RunAsync with a constructed prompt. (5) Saves the result and sends a notification. (6) Repeats up to MaxIterationsPerRun (default 5) per job execution.

## Configuration

| Option | Default | Description |
|---|---|---|
| Enabled | false | Whether autonomous mode is active |
| MaxIterationsPerRun | 5 | Max work items processed per job run |
| PollIntervalSeconds | 30 | Seconds between polls when queue is empty |
| CronExpression | */30 * * * * | Hangfire cron schedule for the recurring job |

## Goal and Work Queue Tools

| Tool | Description |
|---|---|
| create_goal | Create a new goal with name, description, priority, acceptance criteria |
| list_goals | List all goals, optionally filtered by status |
| read_goal | Read full details of a goal |
| update_goal_status | Change goal status (active/completed/paused/cancelled) |
| delete_goal | Delete a goal |
| submit_work_item | Submit a work item to the queue with priority |
| list_work_queue | List items in the work queue |
| cancel_work_item | Cancel a pending work item |
| get_queue_stats | Get work queue statistics (pending, claimed, completed counts) |

## Autonomous Mode Tools

| Tool | Description |
|---|---|
| start_autonomous_mode | Enable autonomous execution, register Hangfire recurring job |
| stop_autonomous_mode | Disable autonomous execution, remove the recurring job |
| get_autonomous_status | Check whether autonomous mode is currently enabled |

# Security

Security is implemented through Microsoft's Agent Governance Toolkit, integrated via AddAiAgentCanvasSecurity().

## What Gets Registered

| Component | Purpose |
|---|---|
| GovernanceKernel | Central governance object with PolicyEngine, AuditEmitter, InjectionDetector |
| GovernanceContextProvider | Pre-LLM scanning for prompt injection in system instructions |
| GovernedMcpGateway | Evaluates tool calls against governance policies (allow/deny) |
| IToolGovernanceWrapper | Wraps tools to enforce governance at execution time |
| ASP.NET Rate Limiter | Fixed-window rate limiting (default 30/minute) |
| Security Headers Middleware | X-Content-Type-Options, X-Frame-Options, Referrer-Policy |

## Governance Policy (YAML)

Rules specify name, scope, action (allow/deny), conditions (equals, in, matches), and reason. The ConflictStrategy (default: DenyOverrides) determines behavior when multiple rules match.

# Adding Custom Agents

Custom agents are lightweight projects under src/Custom/ that define specialized behavior through the seed interface pattern. Seeds declare the agent's components -- persona, guardrails, workflows, context, entities, skills, MCP connections, and tool dependencies. The framework resolves and persists them at startup.

## The Seed Interface Pattern

Nine seed interfaces in Abstractions allow custom agents to declaratively register all their components:

| Interface | Purpose | Key Fields |
|---|---|---|
| IPersonaSeed | Agent identity and instructions | AgentName, Description, Instructions |
| IGuardrailSeed | Safety rules and constraints | Name, Severity, Enabled, Rule |
| IWorkflowSeed | Multi-step procedures | Name, Description, Tags, Content |
| IContextSeed | Domain knowledge documents | Topic, Tags, Content |
| IEntitySeed | Schema/reference definitions | Name, Type, Tags, Content |
| ISkillSeed | Reusable prompt templates | Name, Description, PromptTemplate |
| IMcpConnectionSeed | External MCP server connections | Name, Endpoint, Transport |
| IGoalSeed | Autonomous execution goals | Name, Description, Priority, Content |
| IToolDependencySeed | Required tool declarations | AgentName, RequiredTools |

Seeds never overwrite manual edits -- they only create components if they don't already exist on disk. This means you can seed defaults and then customize them through the chat interface or by editing the markdown files directly.

## Step-by-Step Guide

**Step 1:** Create a project folder under src/Custom/ (e.g., MyAgent)

**Step 2:** Create a .csproj targeting net9.0 with a reference to AiAgentCanvas.Abstractions

**Step 3:** Create a ServiceExtensions class that registers seed implementations:

```
public static class MyAgentServiceExtensions { public static IServiceCollection AddMyAgent(
this IServiceCollection services) { services.AddSingleton&lt;IPersonaSeed&gt;( new PersonaSeed
{ AgentName = "my-agent", Description = "Domain expert", Instructions = "You are a domain
expert..." }); services.AddSingleton&lt;IGuardrailSeed&gt;( new GuardrailSeed { Name =
"safety-rule", Severity = "high", Enabled = true, Rule = "Never disclose internal data" });
services.AddSingleton&lt;IToolDependencySeed&gt;( new ToolDependencySeed("my-agent", new[] {
"tool_a", "tool_b" })); return services; } }
```

**Step 4:** Add ProjectReference in AiAgentCanvas.Web.csproj

**Step 5:** Add to solution: dotnet sln add src/Custom/MyAgent

**Step 6:** Wire up in Program.cs: builder.Services.AddMyAgent();

## HelloWorldAgent Reference

The built-in HelloWorldAgent demonstrates a complete seed implementation with all component types:

| Component | Seed Value |
|---|---|
| Persona | financial-analyst -- Quantitative market analysis expert |
| Guardrail | investment-disclaimer -- Include disclaimers, never give direct advice |
| Workflow | full-stock-analysis -- Multi-step stock analysis procedure |
| Context | financial-analysis-methodology -- Fundamental and technical analysis guide |
| Entity | stock-analysis-report -- Template for structured analysis output |
| Skill | compare-stocks -- Side-by-side stock comparison template |
| Tool Dependencies | stock_quote, stock_history, edgar_company_facts |

# Adding Custom MCP Connections

Custom MCP projects provide tools (data connections, APIs) as class libraries. They use AIFunctionFactory.Create() to wrap .NET methods as LLM-callable tools and register them via IReadOnlyList<AITool>.

## Step-by-Step Guide

**Step 1:** Create a project under src/Custom/ with Microsoft.Extensions.AI and optional HTTP packages

**Step 2:** Create a ToolProvider class with methods decorated with [Description] attributes

**Step 3:** Create ServiceExtensions registering HttpClient, ToolProvider, and IReadOnlyList<AITool>

**Step 4:** Wire up in Program.cs

## MCP.MarketData Reference

The built-in MCP.MarketData project demonstrates the pattern with three tools:

- **edgar_company_facts** -- SEC EDGAR API for company financial data
- **stock_quote** -- Yahoo Finance for real-time stock prices
- **stock_history** -- Historical price data

It registers two named HttpClients ('SEC' and 'Yahoo') with timeouts and AddStandardResilienceHandler() for retry policies (Polly-based). The retry handler automatically recovers from transient connection errors like stale HTTPS connections and socket resets.

## Tool Design Guidelines

- Return JSON strings from tool methods
- Handle errors gracefully -- return error JSON instead of throwing
- Accept and pass CancellationToken
- Use descriptive snake_case names (stock_quote, not StockQuote)
- Keep tools focused -- one action per tool
- Document parameters with [Description] attributes
- Log execution timing for observability